

VERUNUM

TAB5 / R503 Hardware Installation & WebSocket Commissioning Guide

Field guide for the hardware installer and firmware engineer

Platform Verunum institutional attendance & identity	Terminal TAB5-class device
Biometric sensor R503 fingerprint module	Transport WebSocket over LAN/Wi-Fi

Important: This guide is the commissioning contract between the physical terminal and the Verunum platform. Do not guess electrical pinouts or voltage levels. Use the exact TAB5 carrier-board documentation supplied with the hardware.

Version: WebSocket Protocol v1

1. System overview

The terminal communicates with the Verunum backend through a persistent WebSocket. Fingerprint templates remain on the terminal. The server sends and receives the person's UUID, not the biometric template.

```
Admin browser
|
| HTTPS
v
Verunum backend
|
| WebSocket / WSS
v
TAB5 terminal
|
+---- R503 fingerprint sensor
|
+---- Local identity store
```

The operational sequence is:

- Install and power the TAB5 terminal and R503 sensor.
- Connect the terminal to the same network as the Verunum server, or to a network that can route to it.
- Provision the terminal in Verunum using its MAC address and assigned organisation.
- Enter the one-time API key into protected device configuration.
- Open the WebSocket and immediately send the protocol-v1 hello message.
- Wait for hello.ack before processing commands.
- For fingerprint enrollment, receive enroll.start, capture the R503 template locally, then return enroll.result.
- For attendance, match locally and send an attendance event containing the user's UUID.

2. Hardware installation

Use the TAB5 manufacturer's electrical documentation for the exact connector and pin mapping. The Verunum software contract does not define physical pin numbers.

Item	Installer action
TAB5 terminal	Mount securely, provide stable power, and confirm the device boots reliably.
R503 sensor	Connect using the TAB5 carrier's documented UART/power interface. Verify voltage compatibility before powering.
Network	Connect Wi-Fi/Ethernet as supported by the terminal. Confirm the terminal can reach the Verunum host and port.
Enclosure	Leave the fingerprint surface accessible and protect exposed wiring/connectors.

Do not: connect the R503 directly to an unknown 5 V/3.3 V UART until the sensor and carrier-board specifications have been checked.

3. Network and server configuration

- Local development: use ws://HOST:PORT/api/v1/devices/ws.
- Production: use wss://HOST/api/v1/devices/ws behind TLS.

- The terminal must be able to resolve/reach the Verunum server.
- Keep the device API key out of source control and public logs.
- If a browser-based WebSocket is ever used, configure allowed origins with VERUNUM_WS_ORIGINS. Hardware clients normally send no Origin header.

The device can authenticate with the API key in the query string or the X-Device-Key header. Header authentication is preferred where the firmware stack supports it.

4. Provision the terminal

In Verunum, create a pending device registration using the terminal MAC address. The MAC should be normalized as six hexadecimal pairs, for example:

```
AA:BB:CC:DD:EE:FF
```

When the unprovisioned terminal connects with that MAC, Verunum creates the device record and returns a one-time API key in the hello.ack message. Store it securely in the terminal configuration.

Security rule: the server stores only the SHA-256 hash of the device key. The plaintext key is not retained by the platform after provisioning.

5. WebSocket handshake

After the WebSocket connection succeeds, the terminal must send hello immediately:

```
{
  "type": "hello",
  "payload": {
    "protocol_version": 1
  }
}
```

For an already provisioned terminal, the server returns:

```
{
  "type": "hello.ack",
  "status": "authenticated",
  "payload": {
    "device_id": "dev_12",
    "message": "Connected successfully."
  }
}
```

For first-time MAC provisioning, status is **provisioned** and the payload also contains **api_key**. The firmware must store that key securely.

A device that sends any message other than hello as its first JSON frame should treat the connection as failed and reconnect.

6. Fingerprint enrollment flow

```
SERVER -> DEVICE

{
  "type": "enroll.start",
  "request_id": "17",
  "payload": {
    "user_id": "8f14e45f-ea8d-4e4e-9f0c-b1c2d3e4f506",
    "finger_index": 1,
    "timeout_seconds": 30
  }
}
```

Firmware actions:

- Open the fingerprint enrollment screen immediately.
- Prompt the person to place the selected finger on the R503.
- Run the R503 vendor enrollment sequence and verify the template.
- Store the biometric template locally against the supplied user_id.
- Never send the biometric template to Verunum.
- Return enroll.result with the same request_id.

```

DEVICE -> SERVER

{
  "type": "enroll.result",
  "request_id": "17",
  "payload": {
    "user_id": "8f14e45f-ea8d-4e4e-9f0c-b1c2d3e4f506",
    "finger_index": 1,
    "status": "enrolled",
    "error": ""
  }
}

```

The server replies with an ack and updates the person's enrollment status in the dashboard.

7. Attendance flow

The R503 performs the biometric match locally. After a successful match, the firmware maps the local template to the stored user UUID and sends:

```

{
  "type": "attendance",
  "request_id": "att-001",
  "payload": {
    "user_id": "8f14e45f-ea8d-4e4e-9f0c-b1c2d3e4f506",
    "event": "clock_in",
    "timestamp": "2026-09-19T08:04:12Z",
    "method": "fingerprint"
  }
}

```

The server returns an ack containing the attendance record ID and status such as on_time or late.

8. Connection reliability

- Keep the WebSocket open continuously while the terminal is online.
- Use exponential reconnect backoff, for example 1s, 2s, 4s, 8s, 16s, capped around 30s.
- Respond to WebSocket ping frames. The server also accepts JSON ping/pong frames.
- Queue attendance locally while offline and retry after hello.ack.
- If an enroll.start was delivered but no enroll.result was received before disconnect, the server returns that command to pending and replays it after the next successful handshake.
- Do not create duplicate attendance records when retrying; preserve the original request/event identity where possible.

9. Building-site commissioning checklist

Check	Pass	Notes / initials
TAB5 securely mounted	■	

Check	Pass	Notes / initials
R503 connected according to hardware documentation	■	
Stable power verified	■	
Network connectivity verified	■	
Verunum server address configured	■	
Pending device created in Verunum	■	
MAC address matches the terminal	■	
Device receives one-time API key	■	
hello sent immediately after WebSocket connection	■	
hello.ack received	■	
Device appears Online in Verunum	■	
Test person created	■	
Fingerprint enrollment command received	■	
R503 captures and stores local template	■	
enroll.result received by server	■	
Dashboard changes to Enrolled	■	
Fingerprint match produces clock-in event	■	
Clock-out tested	■	
Network interruption tested	■	
Offline attendance queue tested	■	
Reconnect and pending enrollment tested	■	

10. Troubleshooting

Symptom	Check
WebSocket connects but no hello.ack	Confirm the firmware sends {"type":"hello"} immediately after connection and uses protocol_version 1.
Authentication fails	Check API key, server URL, TLS, and that the device record exists.
MAC provisioning fails	Confirm the pending MAC exactly matches the terminal MAC.
Device is online but enrollment does not start	Confirm the selected device is online and that the WebSocket remains connected after hello.ack.
Enrollment completes but dashboard stays pending	Check that enroll.result uses the same request_id and user_id received in enroll.start.
Attendance is not recorded	Check the local fingerprint-to-user UUID mapping and inspect the server ACK payload.

Symptom	Check
Repeated enrollment after reconnect	Check that the device sends enroll.result once and that request_id is preserved.

Field note: If the physical wiring, R503 UART communication, or terminal boot process is failing, resolve that layer before debugging the WebSocket. The WebSocket contract begins once the terminal has a stable network connection.